Meeting 2 SP23

An Introduction to Transactional Memory

@nebu

Outline

Concurrency
- What is Concurrency? (391 Review :/)
- How is it Implemented?

Architectural Support for Concurrency

Transactional Memory
- Key Ideas
- Implementation Highlights
- Simulation Benchmarks

Section 1

Concurrency

Subsection 1

What is Concurrency? (391 Review :/)

Example of Concurrency

- Imagine you have n agents each working on a task, and a global counter for how many tasks have been completed.
- Agent protocol:
  - If task is not done, continue doing task.
  - If task done, increment global task counter, then start on next task.

Difficulties: Race Conditions

- What if two people complete their task at the same time?
- What if you read the counter while it's being changed?
- ...etc.

Sounds Hard, Why Bother?

- Increases throughput.
  - (More work gets done in the same amount of time.)

Subsection 2

How is it Implemented?

Resource Contention

- Insight: the real issue with concurrency is when more than one "thread" needs access to the same data.
  - (For instance, the counter in the first example.)
- We need an "access management system", or "synchronization" for data structures.

Synchronization via Locking

- Associate the shared data structure with a “lock”.
- Only one thread can hold this lock at a time.
- For any thread to access the data structure, it needs to hold the lock.
- Other threads that need access to the data structure wait to grab the lock.

Building a Lock

- Use atomic `test-and-set`: Writes a 1 to a memory location, returns the old value of the location.
- Here's a (simplified) "spinlock":
  - If the previous value was 1, the lock is taken.
  - We wait until the previous value of the lock was 0, which means we managed to set it to 1.

```
volatile int lock = 0;

void critical() {
    while (test_and_set(&lock) == 1);
    // critical section
    lock = 0;  // release lock
}
```

Further Issues

Processes interacting with locks causes problems:

- Deadlock: Two locks, A and B exist. You need both locks to do an operation X. P1 wants to do X and has A, P2 wants to do X and has B. Both wait forever.
- Convoying: P2 is waiting on lock from P1, but P1 dies.
- Priority inversion: higher priority P1 waits on lower priority P2 for lock.
- ...etc.

Lock-free is good! Programmers have to deal less with locks.

Section 2

Architectural Support for Concurrency

Test-and-set

- Generally, dual-port RAMs implement `test-and-set`.
- However, it is expensive:
  - High bus traffic: while a processor waits to get the lock, it keeps initiating bus transactions.
  - Can also be unfair, in the case of a spinlock, the processor busy-waits for the lock (resource starvation).
- Test and test-and-set is an optimization since it reads the cached value of the lock as opposed to trying to grab the lock each time.

Compare-and-swap

- More common, most architectures support CAS.
- `compare_and_swap(addr, old, new)` does the following:
  - If *addr == old, return false.
  - Else *addr = new, return true.
- Allows simpler implementation of locks and semaphores than `test-and-set`, but fundamentally is the same idea.

Towards Lock-Free: LL/SC

- LL: load-linked, SC: store-conditional
- If used sequentially, LL gets the current value of a memory location, and SC stores a value to that location if and only if there were no changes to that location in the interim.
- Low traffic:
  - Load-linked generates no cache coherence traffic.
  - Store-conditional generates no bus traffic if it fails.

How to Use LL/SC (MIPS II)

Note that there is no explicit “lock” — this is lock-free!

```
lockit:
        LL R2, 0(R1)  ; load linked, R2 <= M[R1]
        BNEZ R2, lockit  ; not available, keep spinning
        DADDUI R2, R0, #1  ; put value 1 in R2
        SC R2, 0(R1)  ; store-conditional succeeds if no one
                      ; updated the lock since the last LL
        BEQZ R2, lockit  ; confirm that SC succeeded,
                         ; else keep trying
```

Section 3

Transactional Memory

The Paper

Maurice Herlihy and J. Eliot B. Moss. 1993. Transactional memory: architectural support for lock-free data structures. SIGARCH Comput. Archit. News 21, 2 (May 1993), 289–300. `https://doi.org/10.1145/173682.165164`

(Extends LL/SC to data larger than a word.)

Subsection 1

Key Ideas

A Transaction Is A

- Finite sequence of instructions that is:
  - Sequential: not interleaved with another transaction.
  - Atomic: at the end of the transaction, its changes to memory are either `COMMIT`'ed (published to other processes) or `ABORT`'ed (discarded).
- A single process executes one transaction at a time.

Add the Following Transaction-Memory Instructions

- `LT` (load transactional): read value from memory into private reg.
- `ST` (store transactional): tentatively write private reg value to shared memory location. Note, is only actually updated when `COMMIT`'ed.
- `LTX` (load transactional exclusive): `LT`, with hint that we'll `ST`.

Define

- *Read set* R is the set of locations touched by `LT`.
- *Write set* W is the set of locations touched by `ST` or `LTX`.
- *Data set* $U = R \cup W$.

Transaction-State Instructions

- `COMMIT` tries to make transaction's tentative `ST`'s permanent. It fails if:
  - Locations in U have been modified by some other transaction.
  - Locations in W have been read by some other transaction.
  - Otherwise, it succeeds.
- `ABORT` discards updates to W.
- `VALIDATE` checks that no other transaction changed R, to ensure consistency of the data the current transaction is operating on.

How to Use Transactions

1. Use `LT` and `LTX` to read.
2. `VALIDATE` the reads. If fail, goto step 1.
3. `ST` to modify locations as needed.
4. `COMMIT` the transaction. If fail, goto step 1.

Lock-Free Counter

Process keeps going until it adds `work` counts to `counter`.

```
volatile int counter;

void process(int work) {
  int success = 0;
  while (success < work) {
    ST(&counter, LTX(&counter) + 1);
    if (COMMIT()) {
      success++;
    }
  }
}
```

Subsection 2

Implementation Highlights

Works in Theory, How to Implement?

- Only requires modification to the primary cache of each processor and instructions to talk to it.
- `COMMIT` and `ABORT` are local to primary cache, does not use bus bandwidth.

Multiprocessor Cache Coherence

- Most cache coherence protocols have three kinds of access rights to a memory location:
  - Not immediately accessible by anyone (in-memory only).
  - Non-exclusively accessible by one or more processors.
  - Exclusively accessible by one processor.
- Insight is that these built-in access rights can be used for transaction conflict detection “for free”.

Architectural Changes

- Each processor maintains its usual primary cache and a small fully associative transactional cache.
  - Exclusive: the same entry can't be in both caches.
- Transactional cache holds tentative writes without propagating them to main memory.
- When commit occurs, other processors can snoop the bus and the transaction is committed to memory.

Transactional Cache Line States

- Default cache flags:

Name	Access	Shared?	Modified?
INVALID	none	-	-
VALID	R	Yes	No
DIRTY	R, W	No	Yes
RESERVED	R, W	No	No

- Extra flags for transactional cache:

Name	Meaning
EMPTY	No data
NORMAL	Committed data
XCOMMIT	Discard on commit
XABORT	Discard on abort

Transactional Cache Line States

- Transactional operations cache two entires: one with XCOMMIT and one with XABORT. `ST` writes to the XABORT entry.
- This is to maintain a copy of the old value in case the transaction aborts.
- XCOMMIT is an optimization that prevents write-backs from happening whenever a transaction cache line is dirty.

Bus Cycles

- Add new bus cycles `T_READ` and `T_RFO` for non-exclusive and exclusive access respectively.
- Also add `BUSY`, which aborts the transaction making the request.

Processor State

- Add TACTIVE flag: on whenever first transaction instruction is executed.
- Add TSTATUS: true if active, false if aborted.

How the Protocol Works, Overall

- Let first operation be `LT`.
- Set TACTIVE.
- If transactional cache has a matching XABORT entry, return value.
- If matching NORMAL entry, set to XABORT, copy to another XCOMMIT, and return value.
- If neither, do a `T_READ` on bus. If we get back a BUSY, abort our transaction (TSTATUS = False). Else, setup XABORT and XCOMMIT as usual.
- For `LTX`, use `T_RFO` instead of `T_READ` and set reserved bit in cache line.
- `ST`, similar to `LTX`, but update XABORT's data.

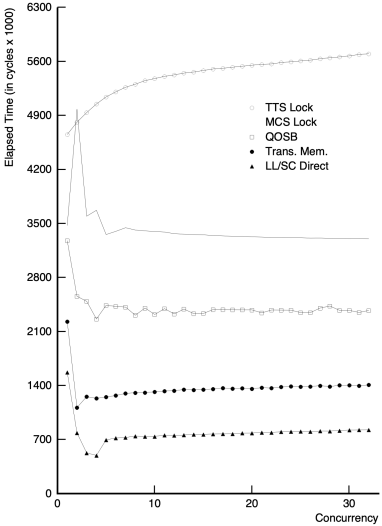
Protocol Working

- `VALIDATE` returns TSTATUS. If false, turns off TACTIVE and turns on TSTATUS.
- `ABORT` discards cache entries then does same as a failed `VALIDATE`.
- `COMMIT` returns TSTATUS, does the same as a failed `VALIDATE`, drops XCOMMIT entries, and sets XABORT entries to NORMAL.

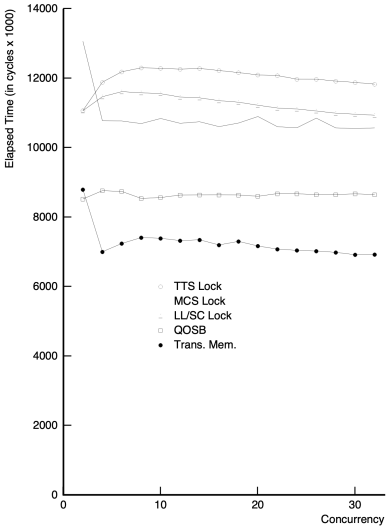
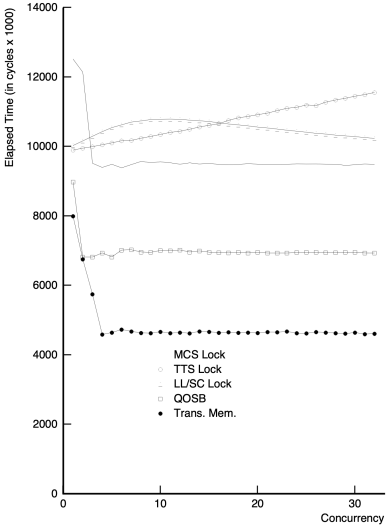
Subsection 3

Simulation Benchmarks

Counting Benchmark



Producer/Consumer Benchmark



Doubly-Linked List Benchmark